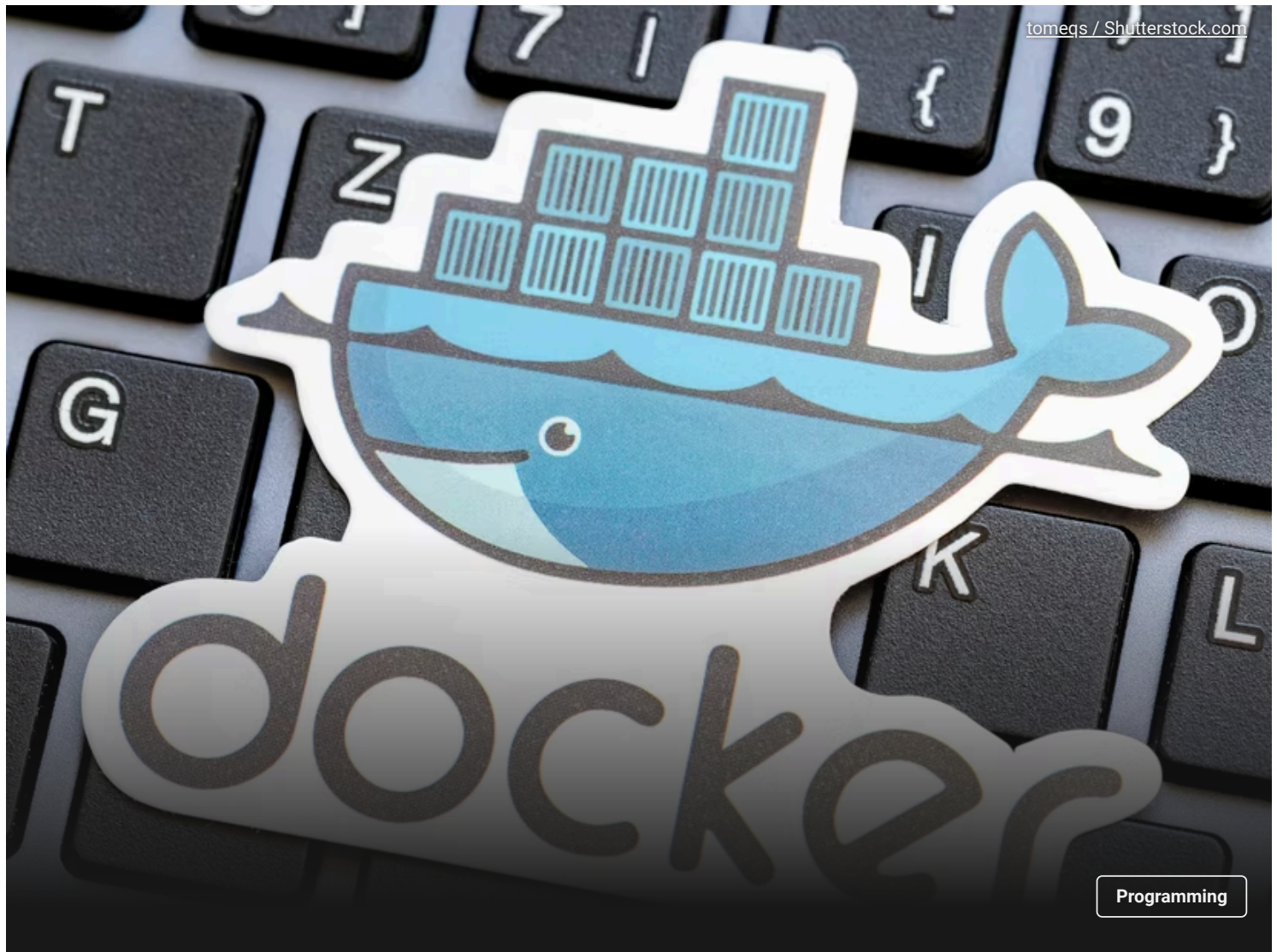


Docker for Beginners: Everything You Need to Know



By James Walker — Aug 31, 2024



 Thread

 Follow

 Share



 How-To Geek



- Docker containers create isolated environments similar to VMs without running a full operating system, enhancing portability and convenience.
- Docker is easy to use and popular for launching applications quickly without impacting your system.
- To use Docker, install it on your platform and build your first Docker image by writing a Dockerfile.

Docker creates packaged applications called containers. Each container provides an isolated environment similar to a virtual machine (VM). Unlike VMs, Docker containers don't run a full operating system. They share your host's kernel and virtualize at a software level.

Link copied to clipboard

Docker Basics

Docker has become a standard tool for software developers and system administrators. It's a neat way to quickly launch applications without impacting the rest of your system. You can spin up a new service with a single **docker run** command.

Containers encapsulate everything needed to run an application, from OS package dependencies to your own source code. You define a container's creation steps as instructions in a Dockerfile. Docker uses the Dockerfile to construct an image.

Images define the software available in containers. This is loosely equivalent to starting a VM with an operating system ISO. If you create an image, any Docker user will be able to launch your app with **docker run**.

How Does Docker Work?

Containers utilize operating system kernel features to provide partially virtualized environments. It's possible to create containers from scratch with commands like chroot. This starts a process with a specified root directory instead of the system root. But using kernel features directly is fiddly, insecure, and error-prone.

Docker is a complete solution for the production, distribution, and use of containers. Modern Docker releases are comprised of several independent components. First, there's the Docker CLI, which is what you interact with in your terminal. The CLI sends commands to a Docker daemon. This can run locally or on a remote host. The daemon is responsible for managing containers and the images they're created from.

The final component is called the container runtime. The runtime invokes kernel features to actually launch containers. Docker is compatible with runtimes that adhere to the OCI specification. This open standard allows for interoperability between different containerization tools.

You don't need to worry too much about Docker's inner workings when you're first getting started. Installing Docker on your system will give you everything you need to build and run containers.

Link copied to clipboard

Why Do So Many People Use Docker?

Containers have become so popular because they solve many common challenges in software development. The ability to containerize once and run everywhere reduces the gap between your development environment and your production servers.

Using containers gives you confidence that every environment is identical. If you have a new team member, they only need to docker run to set up their own development instance. When you launch your service, you can use your Docker image to deploy to production. The live environment will exactly match your local instance, avoiding "it works on my machine" scenarios.

Docker is more convenient than a full-blown virtual machine. VMs are general-purpose tools designed to support every possible workload. By contrast, containers are lightweight, self-sufficient, and better suited to throwaway use cases. As Docker shares the host's kernel, containers have a negligible impact on system performance. Container launch time is almost instantaneous, as you're only starting processes, not an entire operating system.

Getting Started

Docker is available on all popular Linux distributions. It also runs on Windows and macOS. Follow the Docker setup instructions for your platform to get it up and running.

You can check that your installation is working by starting a simple container:

```
docker run hello-world
```

This will start a new container with the basic "hello-world" image. The image emits some output explaining how to use Docker. The container then exits, dropping you back to your terminal.

```
docker run hello-world
```

Link copied to clipboard

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

Creating Images

Once you've run hello-world, you're ready to create your own Docker images. A Dockerfile describes how to run your service by installing required software and copying in files. Here's a simple example using the Apache web server:

```
FROM httpd:latest
RUN echo "LoadModule headers_module modules/mod_headers.so" >> /usr/local/apache2/conf/httpd.conf
COPY .htaccess /var/www/html/.htaccess
COPY index.html /var/www/html/index.html
COPY css/ /var/www/html/css
```

The FROM line defines the base image. In this case, we're starting from the official Apache image. Docker applies the remaining instructions in your Dockerfile on top of the base image.

The RUN stage runs a command within the container. This can be any command available in the container's environment. We're enabling the headers Apache module, which could be used by the .htaccess file to set up routing rules.

The final lines copy the HTML and CSS files in your working directory into the container image. Your image now contains everything you need to run your website.

Link copied to clipboard

Now, you can build the image:

```
docker build -t my-website:v1 .
```

Docker will use your Dockerfile to construct the image. You'll see output in your terminal as Docker runs each of your instructions.

```
jgloor@fedora:~$ sudo docker build -t my-website:v1 .
[+] Building 0.9s (9/9) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile             0.1s
=> => transferring dockerfile: 322B                             0.0s
=> [internal] load metadata for docker.io/library/httpd:latest 0.3s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 28                                    0.0s
=> [internal] load build context                                0.1s
=> => transferring context: 28                                    0.0s
=> [1/5] FROM docker.io/library/httpd:latest@sha256:3f71777bcfac3df3aff5888a2d78c4104501516300b2e7ecb9 0.4s
=> => resolve docker.io/library/httpd:latest@sha256:3f71777bcfac3df3aff5888a2d78c4104501516300b2e7ecb9 0.4s
=> CACHED [2/5] RUN echo "LoadModule headers_module modules/mod_headers.so" >> /usr/local/apache2/conf 0.0s
=> CACHED [3/5] COPY .htaccess /var/www/html/.htaccess         0.0s
=> CACHED [4/5] COPY index.html /var/www/html/index.html       0.0s
```

The `-t` in the command tags your image with a given name (`my-website:v1`). This makes it easier to refer to in the future. Tags have two components, separated by a colon. The first part sets the image name, while the second usually denotes its version. If you omit the colon, Docker will default to using latest as the tag version.

The `.` at the end of the command tells Docker to use the Dockerfile in your local working directory. This also sets the build context, allowing you to use files and folders in your working directory with `COPY` instructions in your Dockerfile.

Once you've created your image, you can start a container using `docker run`:

```
docker run -d -p 8080:80 my-website:v1
```

We're using a few extra flags with `docker run` here. The `-d` flag makes the Docker CLI detach from the container, allowing it to run in the background. A port mapping is defined with `-p`, so

port 8080 on your host maps to port 80 in the container. You should see your web page if you visit `localhost:8080` in your browser.

Link copied to clipboard

Docker images are formed from layers. Each instruction in your Dockerfile creates a new layer. You can use advanced building features to reference multiple base images, discarding intermediary layers from earlier images.

Image Registries

Once you have an image, you can push it to a registry. Registries provide centralized storage so that you can share containers with others. The default registry is Docker Hub.

When you run a command that references an image, Docker first checks whether it's available locally. If it isn't, it will try to pull it from Docker Hub. You can manually pull images with the `docker pull` command:

```
docker pull httpd:latest
```

If you want to publish an image, create a Docker Hub account. Run `docker login` and enter your username and password.

Next, tag your image using your Docker Hub username:

```
docker tag my-image:latest docker-hub-username/my-image:latest
```

Now, you can push your image:

```
docker push docker-hub-username/my-image:latest
```

Other users will be able to pull your image and start containers with it.

You can run your own registry if you need private image storage. Several third-party services also offer Docker registries as alternatives to Docker Hub.

ur Containers

Link copied to clipboard

The Docker CLI has several commands to let you manage your running containers. Here are some of the most useful ones to know:

Listing Containers

`docker ps` shows you all your running containers. Adding the `-a` flag will show stopped containers, too.

```
james@ilmiontdesktop:~$ docker ps
CONTAINER ID   IMAGE
ef361411bc5e   3291fec913ed
6ece9a673033   docker.heron-web.com/brt/brt1119b:latest
b8bb07d57869   docker.heron-web.com/heron-web/heron-web:latest
c0c2425ce928   portainer/portainer-ce
40e101cd9f52   25981d2e449d
3c3e9cb0e591   mysql:5.7
f5499526ecf8   mysql:8.0
24208d8c94ec   docker.heron-web.com/atp/atp0820:master
```

Stopping and Starting Containers

To stop a container, run `docker stop my-container`. Replace `my-container` with the container's name or ID. You can get this information from the `ps` command. A stopped container is restarted with `docker start my-container`.

Containers usually run for as long as their main process stays alive. Restart policies control what happens when a container stops or your host restarts. Pass `--restart always` to `docker run` to make a container restart immediately after it stops.

Getting a Shell

You can run a command in a container using `docker exec my-container my-command`. This is useful when you want to manually invoke an executable that's separate to the container's main process.

Add the `-it` flag if you need interactive access. This lets you drop into a shell by running `docker exec -it my-container sh`.

Monitoring Logs

Link copied to clipboard

Docker automatically collects output emitted to a container's standard input and output streams. The `docker logs my-container` command will show a container's logs inside your terminal. The `--follow` flag sets up a continuous stream so that you can view logs in real time.

```
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET / HTTP/1.1" 200 3039
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET /heron.css HTTP/1.1" 200 8484
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET /fonts/karla-regular.woff2 HTTP/1.1" 200 13444
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET /fonts/karla-bold.woff2 HTTP/1.1" 200 14012
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET /assets/heron.svg HTTP/1.1" 200 534
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET /assets/heron-accent.svg HTTP/1.1" 200 570
172.21.0.1 - - [01/May/2021:10:23:08 +0000] "GET /assets/heron.png HTTP/1.1" 200 7604
172.21.0.1 - - [01/May/2021:10:23:10 +0000] "GET /about HTTP/1.1" 200 2990
172.21.0.1 - - [01/May/2021:10:23:11 +0000] "GET /contact HTTP/1.1" 200 3337
172.21.0.1 - - [01/May/2021:10:23:31 +0000] "GET /privacy HTTP/1.1" 200 3413
172.21.0.1 - - [01/May/2021:10:23:32 +0000] "GET /terms HTTP/1.1" 200 7716
172.21.0.1 - - [01/May/2021:10:23:40 +0000] "GET /careers HTTP/1.1" 200 2457
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /careers/react-developer-0121 HTTP/1.1" 200 6659
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /heron.css HTTP/1.1" 200 8484
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /fonts/karla-regular.woff2 HTTP/1.1" 200 13444
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /fonts/karla-bold.woff2 HTTP/1.1" 200 14012
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /assets/heron.svg HTTP/1.1" 200 534
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /assets/heron-accent.svg HTTP/1.1" 200 570
192.168.0.1 - - [01/May/2021:10:23:42 +0000] "GET /assets/heron.png HTTP/1.1" 200 7604
```

Cleaning Up Resources

Old containers and images can quickly pile up on your system. Use `docker rm my-container` to delete a container by its ID or name.

The command for images is `docker rmi my-image:latest`. Pass the image's ID or full tag name. If you specify a tag, the image won't be deleted until it has no more tags assigned. Otherwise, the given tag will be removed but the image's other tags will remain usable.

Bulk clean-ups are possible using the `docker prune` command. This gives you an easy way to remove all stopped containers and redundant images.

Link copied to clipboard

`docker system prune`**WARNING!** This will remove:

- all stopped containers
- all networks not used by at least one container
- all dangling images
- all dangling build cache

Are you sure you want to continue? [y/N] _

Graphical Management

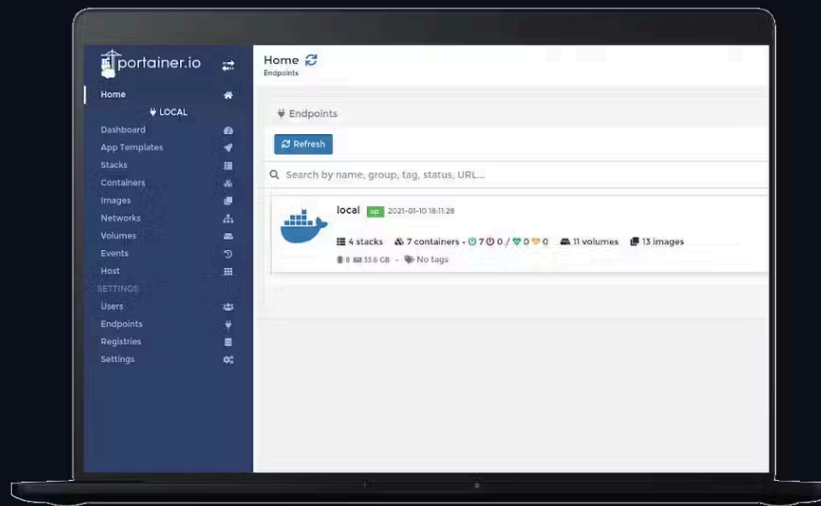
If the terminal's not your thing, you can use third-party tools to set up a graphical interface for Docker. Web dashboards let you quickly monitor and manage your installation. They also help you take remote control of your containers.

Persistent Data Storage

Docker containers are ephemeral by default. Changes made to a container's filesystem won't persist after the container stops. It's not safe to run any form of file storage system in a container started with a basic `docker run` command.

There are a few different approaches to managing persistent data. The most common is to use a Docker Volume. Volumes are storage units that are mounted into container filesystems. Any data in a volume will remain intact after its linked container stops, letting you connect another container in the future.

Link copied to clipboard



Maintaining Security

Dockerized workloads can be more secure than their bare metal counterparts, as Docker provides some separation between the operating system and your services. Nonetheless, Docker is a potential security issue, as it normally runs as root and could be exploited to run malicious software.

If you're only running Docker as a development tool, the default installation is generally safe to use. Production servers and machines with a network-exposed daemon socket should be hardened before you go live.

Audit your Docker installation to identify potential security issues. There are automated tools available that can help you find weaknesses and suggest resolutions. You can also scan individual container images for issues that could be exploited from within.

```
# -----
# Link copied to clipboard
# Docker, Inc. (c) 2015-2021
# Checks for dozens of common best-practices around deploying Docker containers in production.
# Based on the CIS Docker Benchmark 1.3.1.
# -----

Initializing 2021-05-26T15:25:38+01:00

Section A - Check results

[INFO] 1 - Host Configuration
[INFO] 1.1 - Linux Hosts Specific Configuration
[WARN] 1.1.1 - Ensure a separate partition for containers has been created (Automated)
[INFO] 1.1.2 - Ensure only trusted users are allowed to control Docker daemon (Automated)
[INFO] * Users: james
[WARN] 1.1.3 - Ensure auditing is configured for the Docker daemon (Automated)
[WARN] 1.1.4 - Ensure auditing is configured for Docker files and directories - /run/containerd (Automated)
[WARN] 1.1.5 - Ensure auditing is configured for Docker files and directories - /var/lib/docker (Automated)
[WARN] 1.1.6 - Ensure auditing is configured for Docker files and directories - /etc/docker (Automated)
[WARN] 1.1.7 - Ensure auditing is configured for Docker files and directories - docker.service (Automated)
```

Working with Multiple Containers

The docker command only works with one container at a time. You'll often want to use containers in aggregate. [Docker Compose](#) is a tool that lets you define your containers declaratively in a YAML file. You can start them all up with a single command.

This is helpful when your project depends on other services, such as a web backend that relies on a database server. You can define both containers in your `docker-compose.yml` and benefit from streamlined management with [automatic networking](#).

Here's a simple `docker-compose.yml` file:

```
version: "3"
services:
  app:
    image: app-server:latest
    ports:
      - 8000:80
  database:
    image: database-server:latest
    volumes:
      - database-data:/data
```

```
volumes:
```

[Link copied to clipboard](#)

This defines two containers (app and database). A volume is created for the database. This gets mounted to /data in the container. The app server's port 80 is exposed as 8000 on the host. Run `docker-compose up -d` to spin up both services, including the network and volume.

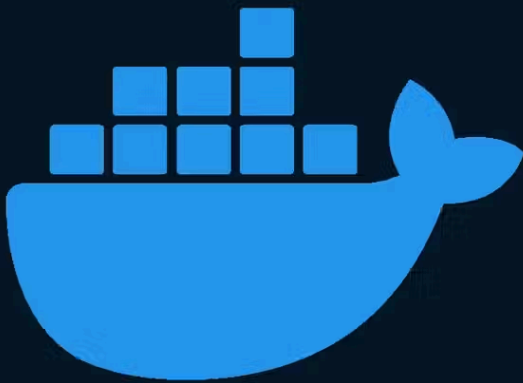
The use of Docker Compose lets you write reusable container definitions that you can share with others. You could commit a `docker-compose.yml` into your version control instead of having developers memorize `docker run` commands.

There are other approaches to running multiple containers, too. Docker App is an emerging solution that provides another level of abstraction. Elsewhere in the ecosystem, Podman is a Docker alternative that lets you create "pods" of containers within your terminal.

Container Orchestration

Docker isn't normally run as-is in production. It's now more common to use an orchestration platform such as Kubernetes or Docker Swarm mode. These tools are designed to handle multiple container replicas, which improves scalability and reliability.

Link copied to clipboard



Docker is only one component in the broader containerization movement. Orchestrators utilize the same container runtime technologies to provide an environment that's a better fit for production. Using multiple container instances allows for rolling updates as well as distribution across machines, making your deployment more resilient to change and outage. The regular docker CLI targets one host and works with individual containers.

A Powerful Platform for Containers

Docker gives you everything you need to work with containers. It has become a key tool for software development and system administration. The principal benefits are increased isolation and portability for individual services.

Getting acquainted with Docker requires an understanding of the basic container and image concepts. You can apply these to create your specialized images and environments that containerize your workloads.